

Implementation Plan

Sleepiness Function Estimation Plan ('estimation_1_sleep.py')

This plan covers the implementation of the depositor sleepiness estimation following Eq-11 and Eq-12 from 'V2.1.0.tex'.

1. Goal Description

The objective is to estimate the depositor sleepiness function $\phi(S_{mt}) = \Upsilon' S_{mt}$ which dictates the fraction of depositor sleepiness.

For regulated deposit types ($k = 1, 2, 3$), rates are exogenous, so endogeneity is not a concern. For endogenously priced types ($k = 4, 5$), we must employ a Control Function (CF) approach to absorb unobserved latent demand.

2. Proposed Changes

[NEW] 'Code/Egan_et_al_2025_Rep/estimation_1_sleep.py'

This script will read 'market_panel.csv' and run various specifications of the model.

****Routine Structure:****

1. **Data Prep**:

- Filter for valid 'deposit_balance' and 'lagged_deposits'.
- Construct the net-return-lagged-deposit base variable:

NR

$$DepL = (1 + risk_{free_qq}(lag) - spread_{qq}(lag)) \times lagged_deposits$$

– Create interactions between NR

$DepL$ and state variables S_{mt} .

– Ensure ' k ' (deposit type) is properly segmented.

2. **First Stage (Eq-11) for $k \in \{4, 5\}$ **:

- Estimate $\rho_{jkmt} = \lambda Z_{jmt} + \nu_{jkmt}$ using OLS with fixed effects.
- Extract the residuals $\hat{\nu}_{jkmt}$.
- For $k \in \{1, 2, 3\}$, force fully set $\hat{\nu}_{jkmt} = 0$.

3. **Second Stage (Eq-12)**:

- Estimate the regression: $Dep_{jkmt} = (\Upsilon' S_{mt}) \times NR$

$$DepL + H(\hat{\nu}_{jkmt}) + F.E. + u_{jkt}$$

– Where $H(\cdot)$ is a cubic polynomial (e.g., $\hat{\nu}, \hat{\nu}^2, \hat{\nu}^3$).

– Fixed Effects: standard specification is 'Conglomerate x Type x Market'.

—

3. Specifications and Discussion

A. State Variables (S_{mt})

We want to explore combinations of state variables. I propose defining baseline, macro, and tech packages:

****Base (S^{Base}) **:** $Constant(intercept)$.

****Macro (S^{Macro}) **:** 'gdp_per_capita', 'cadunico_families_per1000' (income proxy), 'fraction_65plus', 'fraction_young' (a

****Tech/Finance (S^{Tech}) **:** 'pix_users_pf_per1000', 'connections_per100' / 'frac_4g5g', 'branches_per1000'.

****Estimation Variations:****

- ****Spec 1****: $S_{mt} = S^{Base}(Simple constants sleepiness)$
- **Spec2****: $S_{mt} = S^{Base} \cup S^{Macro}(Does local wealth/age affects sleepiness?)$
- **Spec3****: $S_{mt} = S^{Base} \cup S^{Macro} \cup S^{Tech}(Does tech savviness/PIX presence affects sleepiness?)$

B. Instrumental Variables (Z_{jmt}) for $k \in \{4, 5\}$

Including ***all*** the instruments in the tables at once would likely lead to an over-identified model susceptible to weak instrument bias or overfitting in the first stage. We should test a hierarchy of restrictions:

1. ****Classic Operating Shifters (Strict Cost)****:
 - ‘personnel_cost_ratio_lag’, ‘admin_cost_ratio_lag’, ‘tax_cost_ratio_lag’.
 - ***Pros***: Directly follow the theoretical assumption that administrative and wage costs shift deposit-collection spread without altering product quality.
2. ****Wholesale & Capital (Funding Opportunity Cost)****:
 - Above + ‘wholesale_ratio_lag’, ‘indice_basilea_lag’ (Capital ratio).
 - ***Pros***: Captures the bank’s internal need for funding. A bank flushing with wholesale LFTs / strong basel ratio might bid less aggressively for time deposits.
3. ****Hausman IV (Competitive Rival Pricing)****:
 - Above + ‘leave_one_out_mean_spread’ of rivals in the same deposit type/quarter/market.
 - ***Pros***: Usually the strongest predictor of prices. ***Cons***: Requires assuming rival spreads only affect my demand through my spread.
4. ****Full Suite (including ALM and SCR)****:
 - Everything above + ‘credit_assets_ratio_lag’, ‘scr_short_term_share_lag’, etc.
 - ***Pros***: Maximizes first-stage fit. ***Cons***: Might overfit and bleed endogenous latent demand into the predicted spreads.

My recommendation is to use **Instrument Spec 2 or 3** as the baseline for the script’s output tables.

User Review Required

Before I code ‘estimation_1_sleep.py’:

1. Do you agree with the S_{mt} groups (Base, Macro, Tech)?
2. Do you have a preference for which Instrument combinations (1 to 4) we loop over in the script?
3. For fixed effects in the second stage, the text notes ‘Bank x Type x Market’ (Eq-12). Since ‘market_panel.csv’ contains multiple conglomerates, we can use ‘CodConglomeradoPrudencial x deposit_type x mca_code’ and ‘year_quarter’. Is this the exact array of F.E. you want?

Walkthrough

Sleepiness Function Estimation: Proof of Work

Overview

I successfully engineered and implemented ‘estimation_1_sleep.py’ to run the econometrics specification derived from ‘V2.1.0.tex’ (Eq-11 and Eq-12).

What Was Accomplished

****Data Reshaping (Wide to Long)**:** The underlying ‘market_panel.csv’ stores deposit stocks and spreads in a wide format (‘dep_a1’..‘dep_a5’). I implemented logic to pivot this into a long format using ‘deposit_type’ as the primary key suffix, resolving a missing key error.

****Control Function Handling (First Stage)**:** The script systematically regresses endogenously priced types ($k=4, 5$) on the designated instrumental variable blocks to purge latent demand, returning residual polynomials $H(\hat{\nu})$. *Types 1 to 3 are safely assigned null residuals.*

****12-Specification Loop System**:** *The script crosses 4 Instrumental Variable blocks with 3 State Variable packages (Base, Macro, Tech), iterating naturally through the 12 combinations.*

****High-Dimensional Fixed Effects Integration**:** *To safely maneuver the 230,000+ potential dummy columns from the target way Fixed Effect (‘CodConglomeradoPrudencialxdeposit_typexmca_code’), I sidestepped Python’s buggy ‘statsmodels’ dummy*

Validation

A trial execution passed from start to finish.

Spec 3 successfully skipped itself due to ‘leave_one_out_mean_spread’ not being adequately populated in the source data.

The remaining scenarios reliably logged rigorous R^2 values and converged on the exact specifications mapped out in your implementation.

The Issue of Standard Errors: Clustering vs Heteroskedasticity (Egan et al.)

In their framework, Egan et al. frequently cluster standard errors at the ****Bank/Conglomerate**** level when modeling branch or county-level datasets. In ‘estimation_1_sleep.py’, I have matched this by clustering at the ‘CodConglomeradoPrudencial’ level.

1. Clustering at the Conglomerate Level (Current Implementation)

****Theoretical PRO**:** Deposit pricing policies, platform technology (e.g., apps, PIX integrations), and brand marketing are centrally determined. Therefore, unobserved shocks to latent demand ν_{jkm} are highly correlated across different j . ****Applied PRO**:** *If intra-cluster correlation exists, assuming independence will artificially deflate your standard errors. This is a rejection of the null hypothesis. Clustering corrects this downward bias, extending to wider, honest confidence intervals.*

****Theoretical CON**:** *It assumes independence across different conglomerates. If there are massive macroeconomic shocks, this assumption is violated.* ****Applied CON**:** *Clustered standard errors rely on properties asserting the number of clusters $G \rightarrow \infty$. In Brazil, if the bank is biased, or the covariance matrix is not non-positive definite.*

2. Assuming Fully Heteroskedastic (HC1 or HC3)

****Theoretical PRO**:** None.

****Applied PRO**:** It works excellently regardless of the number of conglomerates, bypassing the “few clusters” asymptotic problem. It handles any form of variance scaling gracefully without risking singular covariance matrices.

****Theoretical CON**:** HC1 mathematical assumes there is strictly ****zero correlation**** in the error terms between Itaú’s branch in São Paulo and Itaú’s branch in Rio de Janeiro. This is almost certainly false in banking applications.

****Applied CON**:** Because the “zero correlation” is violated, the calculated standard errors will be vastly too small, tricking the researcher into perceiving “fake” statistically significant results.